



Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques de Tanger (FSTT)

Master IASD – Intelligence Artificielle et Data Science

Sujet :

Développement d'une application Python pour l'analyse de données et le Machine Learning

Réalisé par :

- Imad Elmiri
- Oussama Hafdoune
- Ilias Rafiq

Encadré par :

- Mme Sanae KHALI ISSA

Année universitaire : 2025–2026

Résumé

Ce projet a pour objectif de développer une application informatique polyvalente en utilisant le langage de programmation *Python* afin d'analyser des ensembles de données qualitatives ou quantitatives. L'application intègre plusieurs algorithmes de Machine Learning, notamment la régression linéaire, les arbres de décision, le naïf bayésien, le support vector machine, le k-means, le plus proche voisin, les forêts aléatoires ainsi que les réseaux de neurones artificiels. Elle offre également des fonctionnalités de gestion, de visualisation et de validation des données et des modèles, permettant ainsi une analyse claire et efficace des performances obtenues.

Table des matières

1	Présentation générale du projet	2
1.1	Objectifs du projet	2
1.2	Organisation du rapport	2
2	Conception et architecture de l'application	4
2.1	Vue d'ensemble de l'application	4
2.2	Architecture générale du projet	4
2.3	Description des modules	5
2.3.1	Module principal : Interface utilisateur	5
2.3.2	Module de chargement des données	5
2.3.3	Module d'exploration des données	5
2.3.4	Module de prétraitement des données	5
2.3.5	Module de modélisation	5
2.3.6	Module d'évaluation	6
2.3.7	Module d'exportation	6
2.3.8	Module de documentation	6
3	Algorithmes de Machine Learning implémentés	7
3.1	Algorithmes de régression	7
3.1.1	Régression linéaire	7
3.1.2	Régressions régularisées (Ridge et Lasso)	7
3.1.3	Forêt aléatoire pour la régression	7
3.1.4	Support Vector Regression (SVR)	8
3.1.5	Réseaux de neurones pour la régression	8
3.2	Algorithmes de classification	8

3.2.1	Régression logistique	8
3.2.2	Arbres de décision	8
3.2.3	Naïf Bayésien	8
3.2.4	Support Vector Machine (SVM)	8
3.2.5	K-plus proches voisins (KNN)	8
3.2.6	Forêts aléatoires (Random Forest)	9
3.2.7	Réseaux de neurones artificiels	9
3.3	Algorithmes de clustering	9
3.3.1	K-means	9
3.4	Évaluation des performances des modèles	9
3.4.1	Évaluation des modèles de régression	9
3.4.2	Évaluation des modèles de classification	10
3.4.3	Évaluation des modèles de clustering	10
3.5	Synthèse	10
4	Fonctionnalités de l'application principale app.py	11
4.1	Configuration et initialisation de l'application	11
4.2	Gestion de l'état de l'application	11
4.3	Chargement des données	12
4.4	Exploration des données	12
4.5	Prétraitement des données	12
4.6	Modélisation	13
4.7	Évaluation des modèles	13
4.8	Export des résultats	13
4.9	Documentation intégrée	14
4.10	Synthèse	14
5	Résultats et visualisations	15
5.1	Présentation générale des résultats	15
5.2	Interface graphique de l'application	15

5.3	Chargement des données	16
5.4	Résultats de l'exploration statistique des données	16
5.5	Résultats de l'exploration : distributions des données	17
5.6	Résultats de l'exploration : corrélations des données	17
5.7	Résultats du prétraitement des données	18
5.8	Résultats du prétraitement des données : Normalisation	18
5.9	La page de modélisation(Regression)	18
5.10	Résultats de l'évaluation (Regression)	19
5.11	La page de modélisation(Classification)	21
5.12	Résultats de l'évaluation (Classification)	21
5.13	La page de modélisation (Clustering)	22
5.14	Résultats du évaluation (clustering)	23
5.15	Exportation des résultats du modèle	23
5.16	Documentation et tutoriels	24
5.16.1	Tutoriels des algorithmes	24
5.16.2	Documentation technique	24
6	Conclusion générale et perspectives	26
6.1	Conclusion générale	26
6.2	Perspectives d'amélioration	26

Introduction

Dans le cadre du Master Intelligence Artificielle et Data Science (IASD) et du module Programmation Avancée, ce projet a pour objectif principal de mettre en pratique la programmation avancée en Python à travers le développement d'une application informatique interactive.

Le projet vise à illustrer la structuration du code, la modularité, la gestion des données et l'intégration de bibliothèques Python.

Le Machine Learning est utilisé comme un cas d'usage applicatif, permettant de démontrer l'utilisation de Python dans l'analyse et le traitement de données qualitatives et quantitatives, sans constituer l'objectif principal du projet.

L'application développée permet à l'utilisateur de charger, explorer, traiter des données et de visualiser les résultats à travers une interface conviviale.

Chapitre 1 : Présentation générale du projet

Ce chapitre présente une vue d'ensemble du projet, en mettant en évidence ses objectifs principaux, son architecture globale ainsi que les fonctionnalités offertes par l'application développée.

1.1 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de développer une application Python **modulaire et interactive**, mettant en pratique les concepts de **programmation avancée**, à travers un cas d'usage lié à l'analyse de données. Le Machine Learning est utilisé **comme un support applicatif** afin d'illustrer l'intégration de bibliothèques Python spécialisées, sans constituer l'objectif principal du projet.

Les objectifs spécifiques du projet sont les suivants :

- Mettre en pratique les concepts de programmation avancée en Python ;
- Concevoir une application structurée, maintenable et extensible ;
- Permettre l'importation, la préparation et la transformation des données ;
- Intégrer des algorithmes de Machine Learning comme fonctionnalités applicatives ;
- Visualiser les résultats de manière claire et intuitive ;
- Offrir la possibilité d'exporter les résultats et les modèles entraînés.

1.2 Organisation du rapport

Ce rapport est structuré comme suit :

- Le premier chapitre présente une vue d'ensemble du projet ainsi que ses objectifs et l'organisation générale du rapport ;
- Le deuxième chapitre est consacré à la conception et à l'architecture de l'application, en détaillant les différents modules et leur rôle dans le pipeline de Machine Learning ;

-
- Le troisième chapitre décrit les algorithmes de Machine Learning implémentés, incluant les modèles de régression, de classification et de clustering, ainsi que l'évaluation de leurs performances ;
 - Le quatrième chapitre présente les fonctionnalités de l'application principale `app.py`, en détaillant le déroulement des différentes étapes allant du chargement des données jusqu'à l'exportation des résultats ;
 - Le cinquième chapitre est consacré à la présentation des résultats obtenus et à leur visualisation à travers l'interface graphique de l'application ;
 - Enfin, le dernier chapitre conclut le rapport et présente les perspectives d'amélioration et d'évolution du projet.

Chapitre 2 : Conception et architecture de l'application

2.1 Vue d'ensemble de l'application

L'application développée est une application Python interactive intégrant des fonctionnalités de Machine Learning à des fins applicatives à l'aide du framework Streamlit. Elle permet à l'utilisateur d'importer des données, de les explorer, de les prétraiter, d'appliquer différents algorithmes de Machine Learning et d'évaluer les performances des modèles obtenus.

L'application est conçue selon une architecture modulaire afin d'assurer une meilleure lisibilité du code, une facilité de maintenance et une possibilité d'évolution future. Chaque fonctionnalité principale est implémentée dans un module indépendant.

2.2 Architecture générale du projet

L'application est structurée autour des fichiers principaux suivants :

- **app.py** : gestion de l'interface Streamlit et orchestration du flux de l'application ;
- **data_loader.py** : chargement des données ;
- **preprocessing.py** : prétraitement des données ;
- **models.py** : définition, configuration et entraînement des modèles de Machine Learning ;
- **evaluation.py** : évaluation des performances et visualisation des résultats ;

Chaque fichier correspond à une tâche bien définie dans le pipeline de traitement des données et d'apprentissage automatique.

2.3 Description des modules

Cette section décrit les différents modules fonctionnels de l'application, chacun correspondant à une étape précise du pipeline de Machine Learning. Ces modules sont accessibles via le menu de navigation et permettent de structurer le traitement des données de manière progressive et cohérente.

2.3.1 Module principal : Interface utilisateur

Le module `app.py` constitue le point d'entrée principal de l'application. Il est responsable de la gestion de l'interface graphique développée avec le framework **Streamlit**, ainsi que de la navigation entre les différentes étapes de l'application. Ce module coordonne les interactions entre les autres modules et assure le bon déroulement du pipeline de Machine Learning.

2.3.2 Module de chargement des données

Ce module permet à l'utilisateur d'importer les jeux de données à analyser. Les données peuvent être chargées depuis des fichiers locaux, tels que des fichiers CSV ou Excel. Une fois importées, les données sont affichées afin de vérifier leur structure et leur contenu.

2.3.3 Module d'exploration des données

Le module d'exploration offre des outils de visualisation et d'analyse descriptive des données. Il permet d'observer les distributions des variables, d'identifier les valeurs aberrantes et d'analyser les relations entre les différentes caractéristiques du jeu de données.

2.3.4 Module de prétraitement des données

Ce module est dédié à la préparation des données avant la phase de modélisation. Il permet notamment de traiter les valeurs manquantes, d'encoder les variables catégorielles et de normaliser les variables numériques afin d'améliorer la qualité des données.

2.3.5 Module de modélisation

Le module de modélisation permet de sélectionner la variable cible, de choisir le type de problème à traiter et de sélectionner un algorithme de Machine Learning

approprié. L'utilisateur peut ensuite entraîner le modèle à partir des données préparées.

2.3.6 Module d'évaluation

Ce module permet d'évaluer les performances des modèles entraînés à l'aide de métriques adaptées au type de problème traité. Les résultats sont présentés sous forme d'indicateurs numériques et de visualisations facilitant leur interprétation.

2.3.7 Module d'exportation

Le module d'exportation permet de sauvegarder les résultats obtenus, notamment les métriques de performance, les prédictions générées par le modèle ainsi que le modèle entraîné, afin de permettre leur réutilisation ultérieure.

2.3.8 Module de documentation

Ce module fournit des tutoriels et une documentation technique pour accompagner l'utilisateur dans l'utilisation de l'application. Il permet également de présenter l'architecture générale et la description des modules pour les développeurs.

Chapitre 3 : Algorithmes de Machine Learning implémentés

Ce chapitre présente les principaux algorithmes de Machine Learning intégrés dans l'application développée. Ces algorithmes couvrent les trois grandes catégories de l'apprentissage automatique : la régression, la classification et le clustering.

3.1 Algorithmes de régression

Les algorithmes de régression sont utilisés lorsque la variable cible est continue. Ils permettent de modéliser la relation entre les variables explicatives et la variable à prédire.

3.1.1 Régression linéaire

La régression linéaire permet de prédire une variable continue à partir d'une combinaison linéaire des variables explicatives. Elle peut être utilisée aussi bien pour des cas simples que multiples selon le nombre de variables.

3.1.2 Régressions régularisées (Ridge et Lasso)

Les régressions Ridge et Lasso sont des extensions de la régression linéaire intégrant des mécanismes de régularisation afin de limiter le sur-apprentissage et d'améliorer la capacité de généralisation du modèle.

3.1.3 Forêt aléatoire pour la régression

La forêt aléatoire pour la régression repose sur l'agrégation de plusieurs arbres de décision. Elle permet de capturer des relations non linéaires et offre une bonne robustesse face au bruit.

3.1.4 Support Vector Regression (SVR)

Le Support Vector Regression est une méthode de régression basée sur le principe des machines à vecteurs de support. Il est capable de modéliser des relations complexes grâce à l'utilisation de fonctions noyau.

3.1.5 Réseaux de neurones pour la régression

Les réseaux de neurones appliqués à la régression permettent d'apprendre des relations non linéaires complexes entre les variables d'entrée et la variable cible.

3.2 Algorithmes de classification

Les algorithmes de classification sont utilisés lorsque la variable cible est catégorielle. Leur objectif est d'affecter chaque observation à une classe prédéfinie.

3.2.1 Régression logistique

La régression logistique est un modèle probabiliste utilisé principalement pour les problèmes de classification binaire. Elle estime la probabilité d'appartenance à une classe à partir des variables explicatives.

3.2.2 Arbres de décision

Les arbres de décision sont des modèles interprétables basés sur des règles de décision successives. Ils sont faciles à comprendre et à visualiser.

3.2.3 Naïf Bayésien

Le classifieur naïf bayésien est un modèle probabiliste fondé sur le théorème de Bayes et l'hypothèse d'indépendance conditionnelle entre les variables.

3.2.4 Support Vector Machine (SVM)

Le Support Vector Machine cherche à déterminer une frontière de décision optimale maximisant la marge entre les classes, avec la possibilité d'utiliser des fonctions noyau pour traiter des cas non linéaires.

3.2.5 K-plus proches voisins (KNN)

L'algorithme K-plus proches voisins classe une observation en fonction des classes majoritaires parmi ses k voisins les plus proches selon une mesure de distance.

3.2.6 Forêts aléatoires (Random Forest)

Les forêts aléatoires combinent plusieurs arbres de décision afin d'améliorer la robustesse du modèle et de réduire le risque de sur-apprentissage.

3.2.7 Réseaux de neurones artificiels

Les réseaux de neurones artificiels sont constitués de couches de neurones interconnectés capables d'apprendre des relations complexes et non linéaires à partir des données.

3.3 Algorithmes de clustering

Les algorithmes de clustering relèvent de l'apprentissage non supervisé. Ils visent à regrouper automatiquement les données en clusters homogènes sans utiliser de variable cible.

3.3.1 K-means

L'algorithme K-means partitionne les données en k groupes distincts en minimisant la distance entre les points d'un même cluster et leur centroïde.

3.4 Évaluation des performances des modèles

L'évaluation des performances constitue une étape essentielle du processus de Machine Learning. Elle permet de mesurer la qualité des modèles entraînés et de comparer différentes approches afin de sélectionner la plus adaptée aux données étudiées.

Dans l'application développée, l'évaluation est réalisée automatiquement après l'entraînement du modèle, à l'aide de métriques quantitatives et de visualisations graphiques adaptées au type de problème traité.

3.4.1 Évaluation des modèles de régression

Pour les modèles de régression, l'objectif est de mesurer l'écart entre les valeurs prédites et les valeurs réelles. Les métriques utilisées sont :

- l'erreur quadratique moyenne (MSE) ;
- la racine de l'erreur quadratique moyenne (RMSE) ;
- le coefficient de détermination (R^2).

Des graphiques comparant les valeurs réelles aux valeurs prédites sont également générés afin d'évaluer visuellement la qualité de l'ajustement du modèle.

3.4.2 Évaluation des modèles de classification

Pour les modèles de classification, l'évaluation repose sur la comparaison entre les classes réelles et les classes prédites. Les principales métriques utilisées sont :

- l'accuracy ;
- la précision ;
- le rappel ;
- le F1-score.

Une matrice de confusion est également utilisée afin d'analyser les prédictions correctes et incorrectes pour chaque classe.

3.4.3 Évaluation des modèles de clustering

Dans le cas du clustering, l'évaluation est réalisée sans variable cible. Les performances sont mesurées à l'aide de métriques internes, notamment :

- l'inertie ;
- le coefficient de silhouette.

Des visualisations graphiques des clusters sont générées afin de représenter la structure des groupes formés par l'algorithme et d'en faciliter l'interprétation.

3.5 Synthèse

Les algorithmes intégrés dans l'application offrent une diversité de méthodes permettant d'analyser différents types de données. L'évaluation systématique des performances permet de comparer les modèles et de sélectionner celui le plus adapté en fonction du problème étudié et des caractéristiques des données.

Chapitre 4 : Fonctionnalités de l'application principale `app.py`

Ce chapitre décrit les différentes fonctionnalités implémentées dans le fichier `app.py`, qui constitue le point d'entrée principal de l'application. Ce fichier assure la gestion de l'interface utilisateur, la navigation entre les différentes étapes du pipeline de Machine Learning et la coordination entre les différents modules du projet.

L'application est développée à l'aide du framework **Streamlit**, permettant la création d'une interface web interactive dédiée à l'analyse de données et au Machine Learning.

4.1 Configuration et initialisation de l'application

Au démarrage de l'application, plusieurs bibliothèques essentielles sont importées, notamment **pandas**, **numpy**, **matplotlib**, **seaborn** et **scikit-learn**. Les modules internes du projet sont également chargés afin d'assurer la communication entre le chargement des données, le prétraitement, la modélisation et l'évaluation.

L'interface est configurée à l'aide de la fonction `st.set_page_config()`, permettant de définir le titre de l'application, l'icône ainsi que la disposition générale de la page.

Un thème graphique personnalisé est ensuite appliqué à l'aide de styles CSS afin d'améliorer l'ergonomie et l'aspect visuel de l'application.

4.2 Gestion de l'état de l'application

L'application utilise le mécanisme `st.session_state` de Streamlit pour conserver les données et les objets importants tout au long de la navigation. Cela permet notamment de stocker :

- le jeu de données chargé ;
- une copie du jeu de données original ;

-
- le modèle de Machine Learning entraîné ;
 - l'état d'entraînement du modèle.

Cette approche garantit une expérience utilisateur fluide sans perte d'informations lors du changement de section.

4.3 Chargement des données

Cette fonctionnalité permet à l'utilisateur d'importer des données à partir de différentes sources :

- fichiers locaux (CSV ou Excel) ;
- URL distante ;
- jeux de données d'exemple (Iris, Diabetes).

Les données sont chargées sous forme de **DataFrame** pandas et stockées dans l'état de session. Un aperçu des données, leurs types ainsi que des statistiques descriptives sont automatiquement affichés.

4.4 Exploration des données

La phase d'exploration permet d'analyser les caractéristiques principales du jeu de données. Elle inclut :

- des statistiques descriptives ;
- la visualisation des distributions des variables numériques ;
- l'analyse des corrélations à l'aide de matrices de corrélation.

Ces visualisations aident l'utilisateur à mieux comprendre la structure des données avant toute phase de modélisation.

4.5 Prétraitement des données

Le module de prétraitement intégré dans **app.py** permet de préparer les données pour l'apprentissage automatique. Les fonctionnalités proposées incluent :

- la gestion des valeurs manquantes ;
- la suppression des doublons ;
- l'encodage des variables catégorielles ;

— la normalisation des variables numériques.

Chaque transformation est appliquée de manière interactive, avec un retour visuel immédiat sur les données modifiées.

4.6 Modélisation

La phase de modélisation permet à l'utilisateur de :

- sélectionner la variable cible ;
- choisir le type de problème (régression, classification ou clustering) ;
- sélectionner les variables explicatives ;
- choisir un algorithme de Machine Learning ;
- configurer les paramètres du modèle.

Les modèles sont créés dynamiquement à l'aide de la classe `MLModelManager`. L'entraînement, la prédiction et la validation croisée sont déclenchés directement depuis l'interface.

4.7 Évaluation des modèles

Une fois le modèle entraîné, l'application propose une évaluation complète de ses performances. Les métriques affichées dépendent du type de problème :

- régression : MSE, RMSE, MAE et coefficient R^2 ;
- classification : accuracy, précision, rappel, F1-score et matrice de confusion ;
- clustering : inertie et coefficient de silhouette.

Des visualisations graphiques sont également générées afin de faciliter l'interprétation des résultats, telles que les graphiques prédictions/réel ou les représentations des clusters.

4.8 Export des résultats

Cette fonctionnalité permet à l'utilisateur d'exporter :

- les métriques de performance (CSV ou JSON) ;
- les résultats de prédiction ;
- le modèle entraîné sous forme de fichier sérialisé.

Ces exports facilitent la réutilisation des résultats et l'intégration dans d'autres projets.

4.9 Documentation intégrée

L'application intègre une section de documentation permettant d'expliquer le fonctionnement global de l'outil ainsi que les principes des algorithmes de Machine Learning implémentés. Des tutoriels et guides d'utilisation sont accessibles directement depuis l'interface.

4.10 Synthèse

Le fichier `app.py` constitue le cœur de l'application. Il assure l'orchestration complète du pipeline de Machine Learning, depuis le chargement des données jusqu'à l'export des résultats, tout en offrant une interface interactive, intuitive et adaptée aux utilisateurs non experts.

Chapitre 5 : Résultats et visualisations

5.1 Présentation générale des résultats

Ce chapitre présente les résultats obtenus à partir de l'application développée, ainsi que leur visualisation à travers l'interface graphique. Les résultats affichés permettent d'illustrer concrètement le fonctionnement de l'application et les performances des différents modèles de Machine Learning implémentés.

Les visualisations présentées dans ce chapitre sont générées automatiquement par l'application à l'aide des bibliothèques **matplotlib** et **seaborn**, et sont accessibles via l'interface Streamlit.

5.2 Interface graphique de l'application

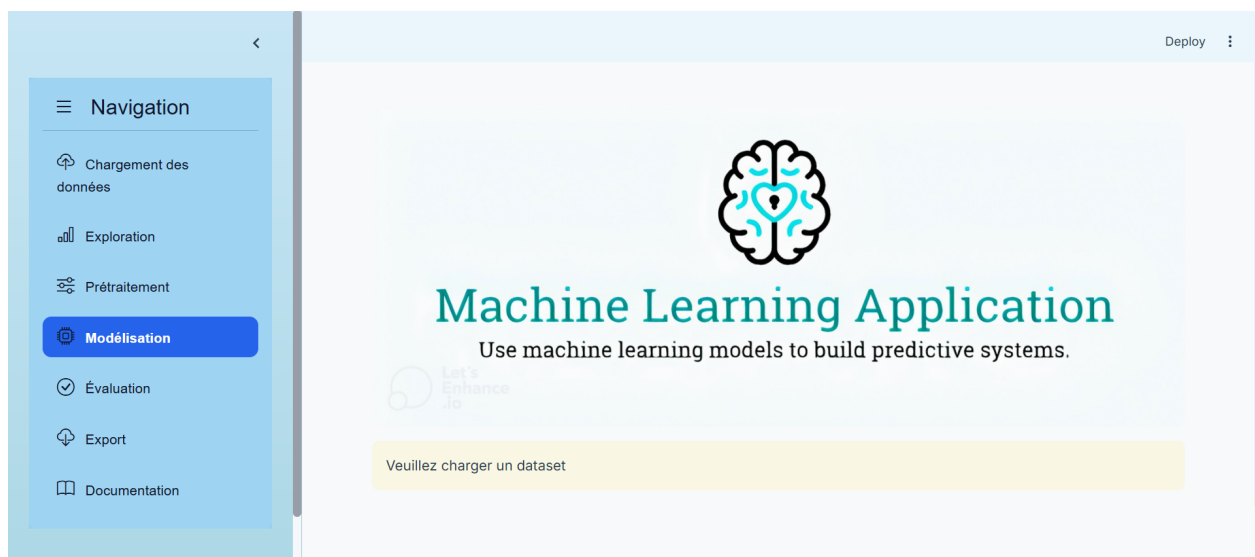


FIGURE 5.1 – Interface graphique principale de l'application Streamlit

5.3 Chargement des données

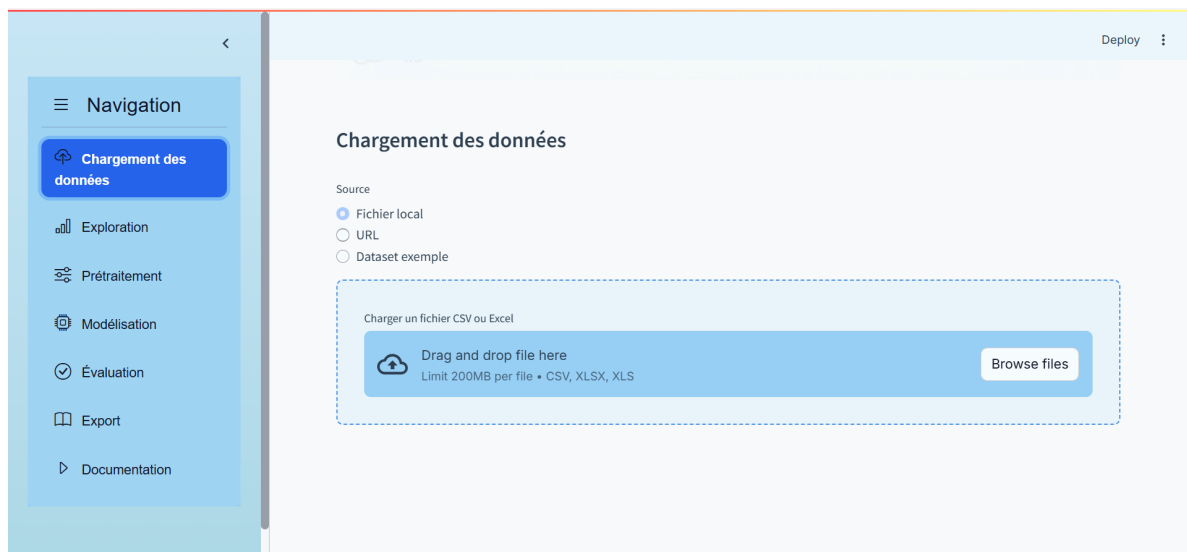


FIGURE 5.2 – Visualisation exploratoire des données

5.4 Résultats de l'exploration statistique des données



FIGURE 5.3 – Visualisation exploratoire statistique des données

5.5 Résultats de l'exploration : distributions des données

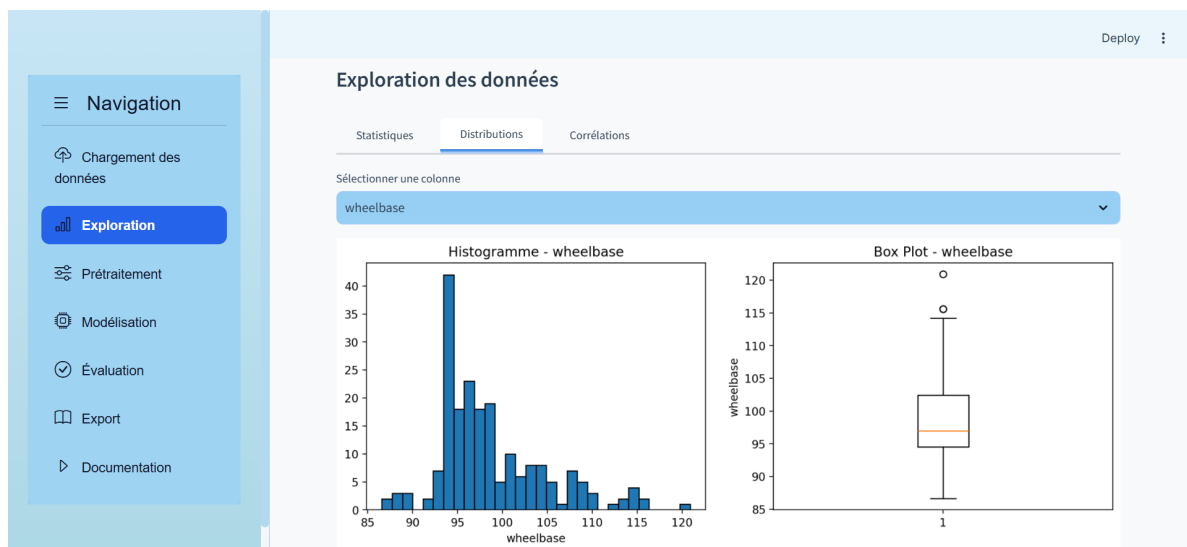


FIGURE 5.4 – Visualisation exploratoire distribution des données

5.6 Résultats de l'exploration : correlations des données

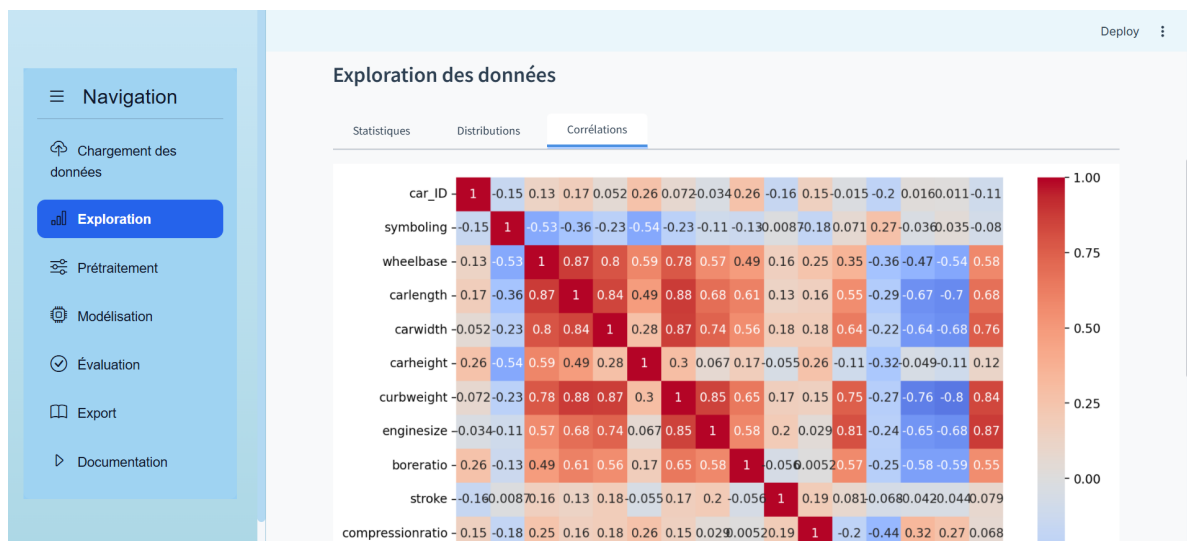


FIGURE 5.5 – Visualisation exploratoire correlations des données

5.7 Résultats du prétraitement des données

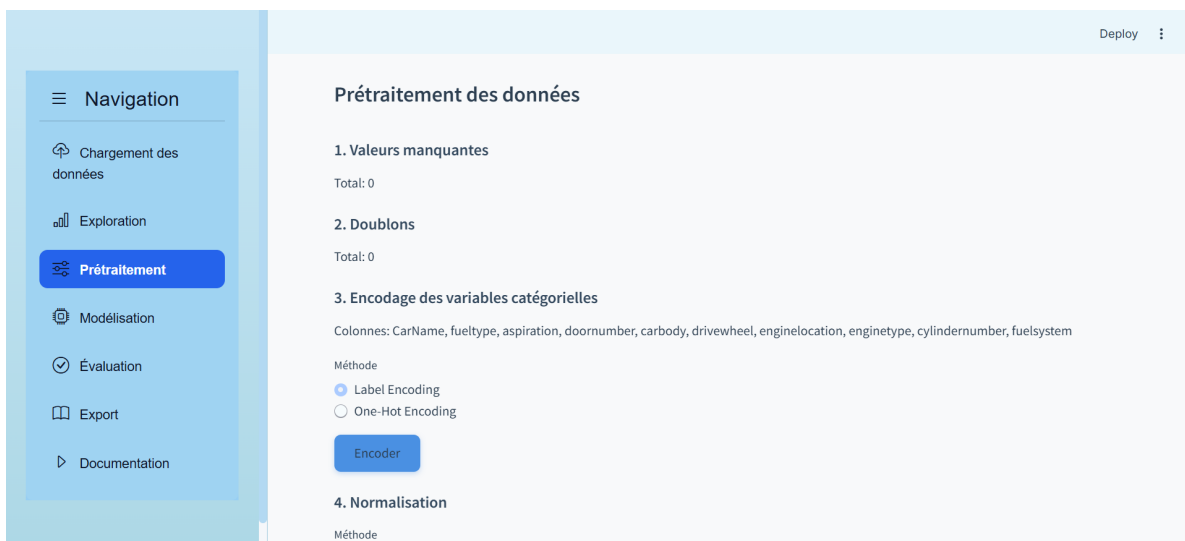


FIGURE 5.6 – Résultats du prétraitement des données

5.8 Résultats du prétraitement des données : Normalisation

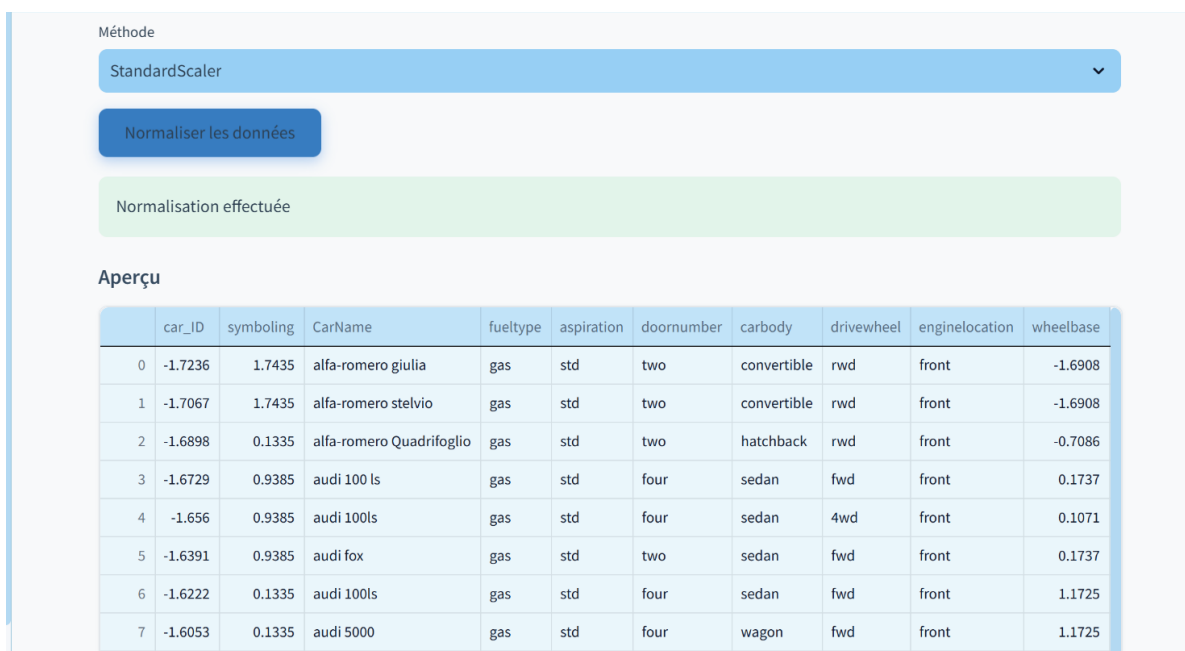


FIGURE 5.7 – Normalisation des données

5.9 La page de modélisation(Regression)

Une démarche similaire a été appliquée à un problème de régression à l'aide du dataset *CarPrice*, permettant de prédire le prix des véhicules à partir de leurs caractéristiques. Cette expérimentation illustre l'utilisation des algorithmes de régression au sein de l'application Python.

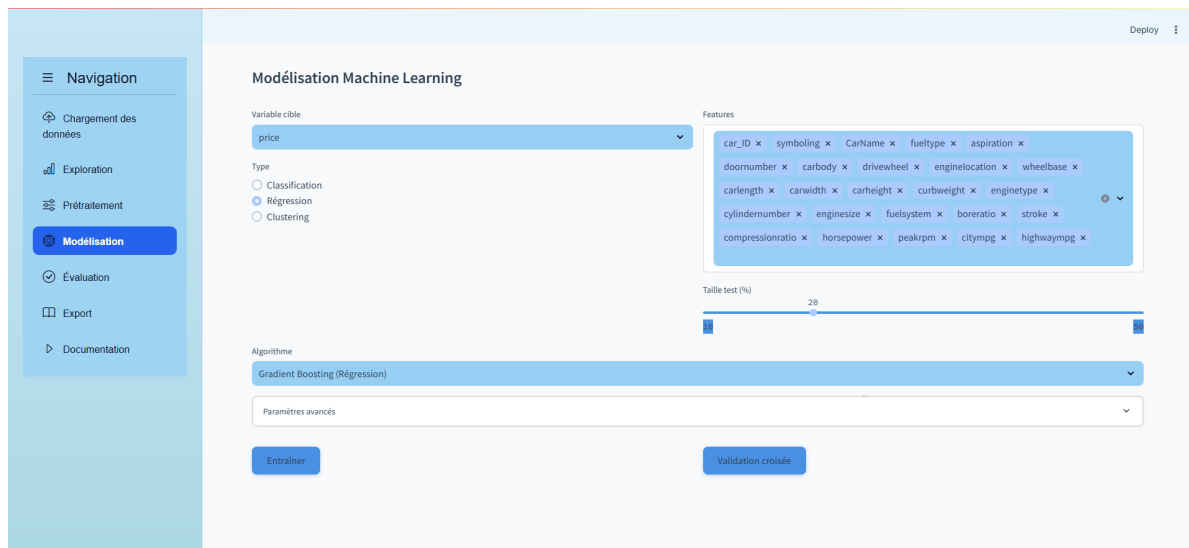


FIGURE 5.8 – La page de modélisation(Regression)

5.10 Résultats de l'évaluation (Regression)

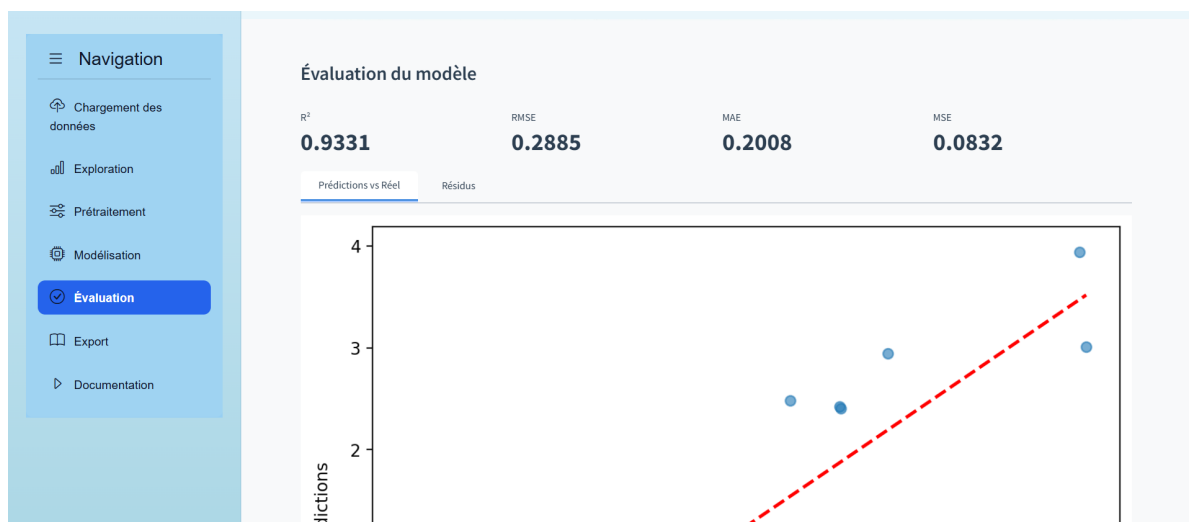


FIGURE 5.9 – Résultats de modèle de régression : Gradient boosting

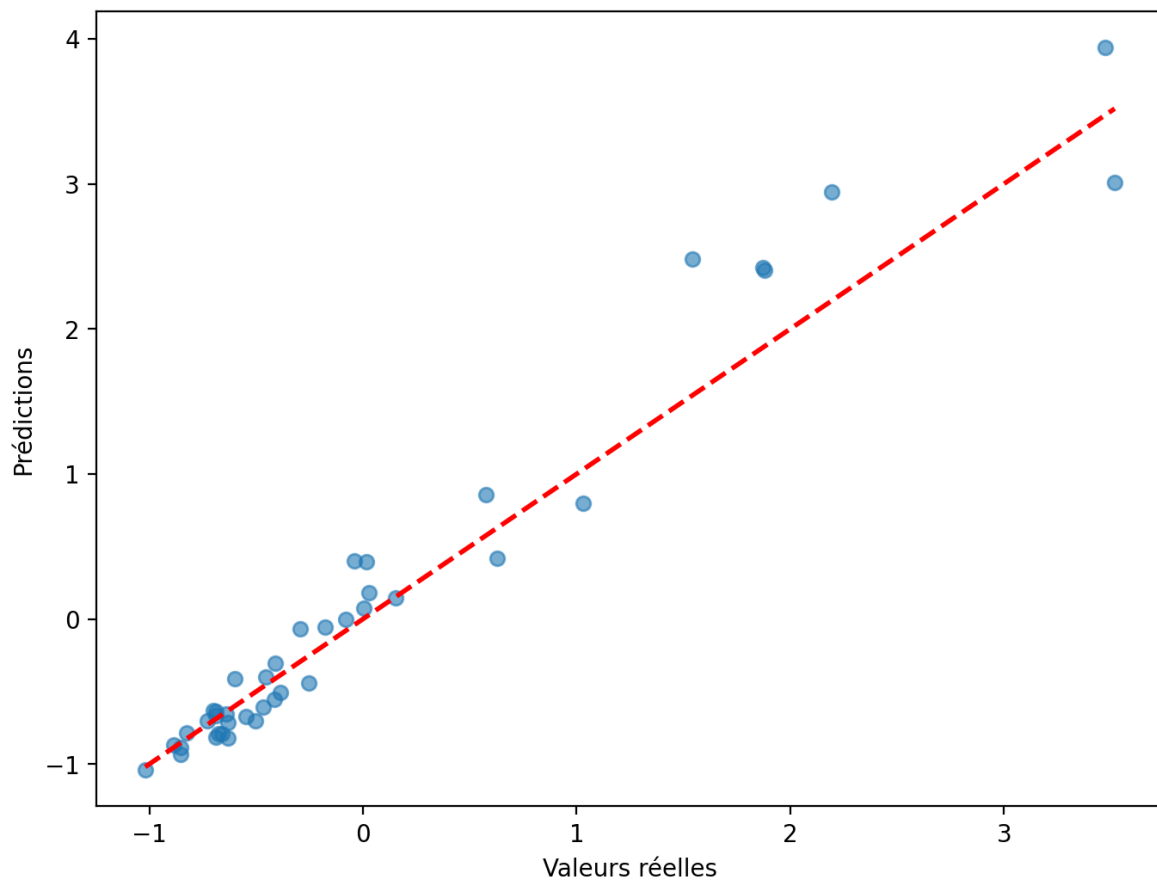


FIGURE 5.10 – Résultats de modèle de régression : Gradient boosting

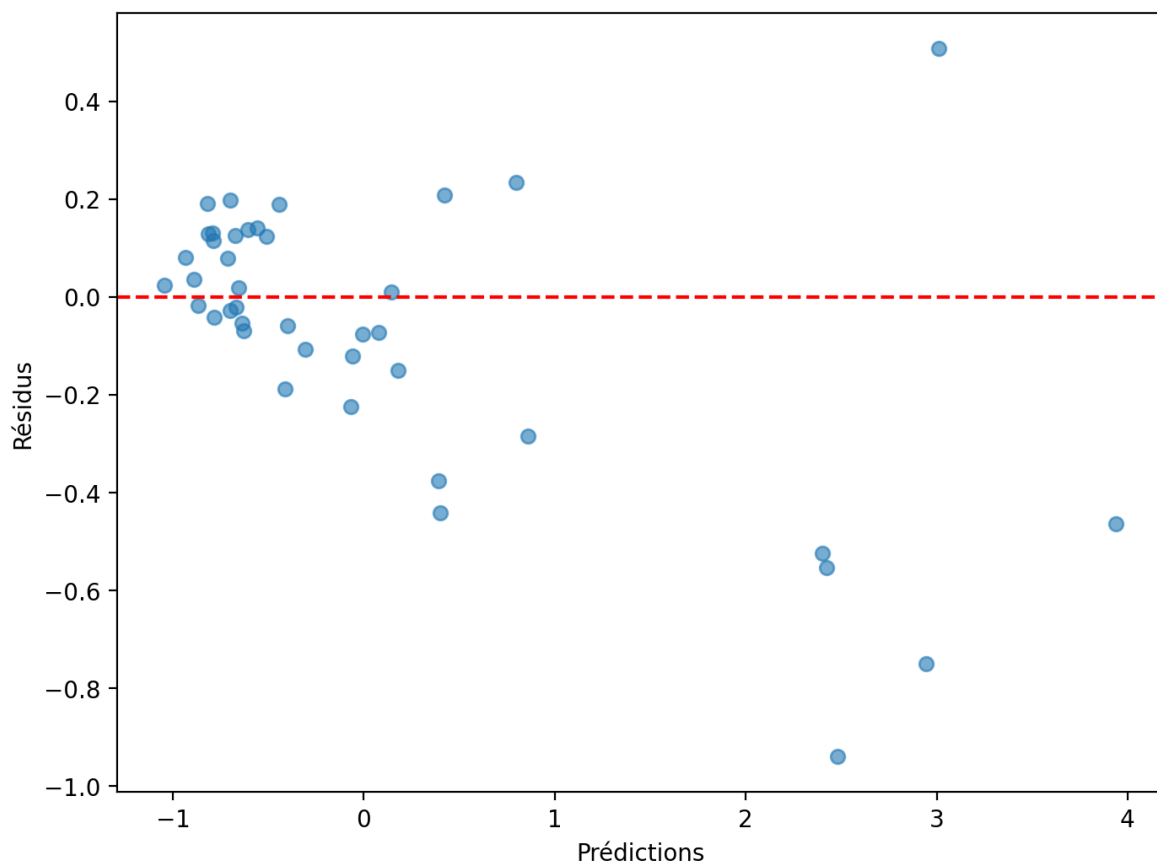


FIGURE 5.11 – Résultats de modèle de régression : Gradient boosting

5.11 La page de modélisation(Classification)

Une démarche similaire a été appliquée à un problème de classification à l'aide du dataset *Diabetes*, conduisant aux résultats présentés ci-dessous.

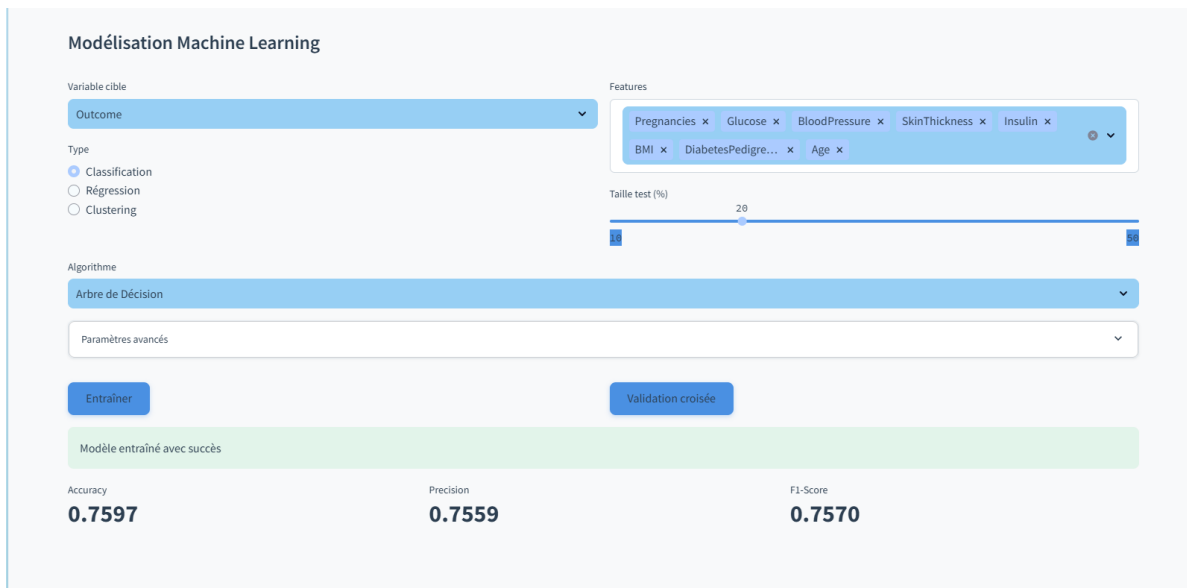


FIGURE 5.12 – La page de modélisation(Classification)

5.12 Résultats de l'évaluation (Classification)

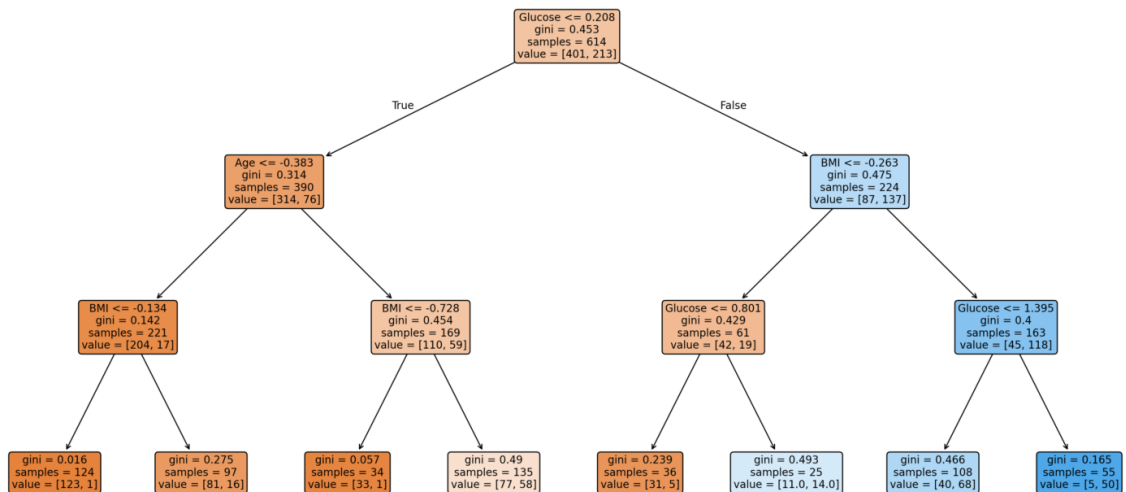


FIGURE 5.13 – Résultats de modèle de classification : Arbre de décision

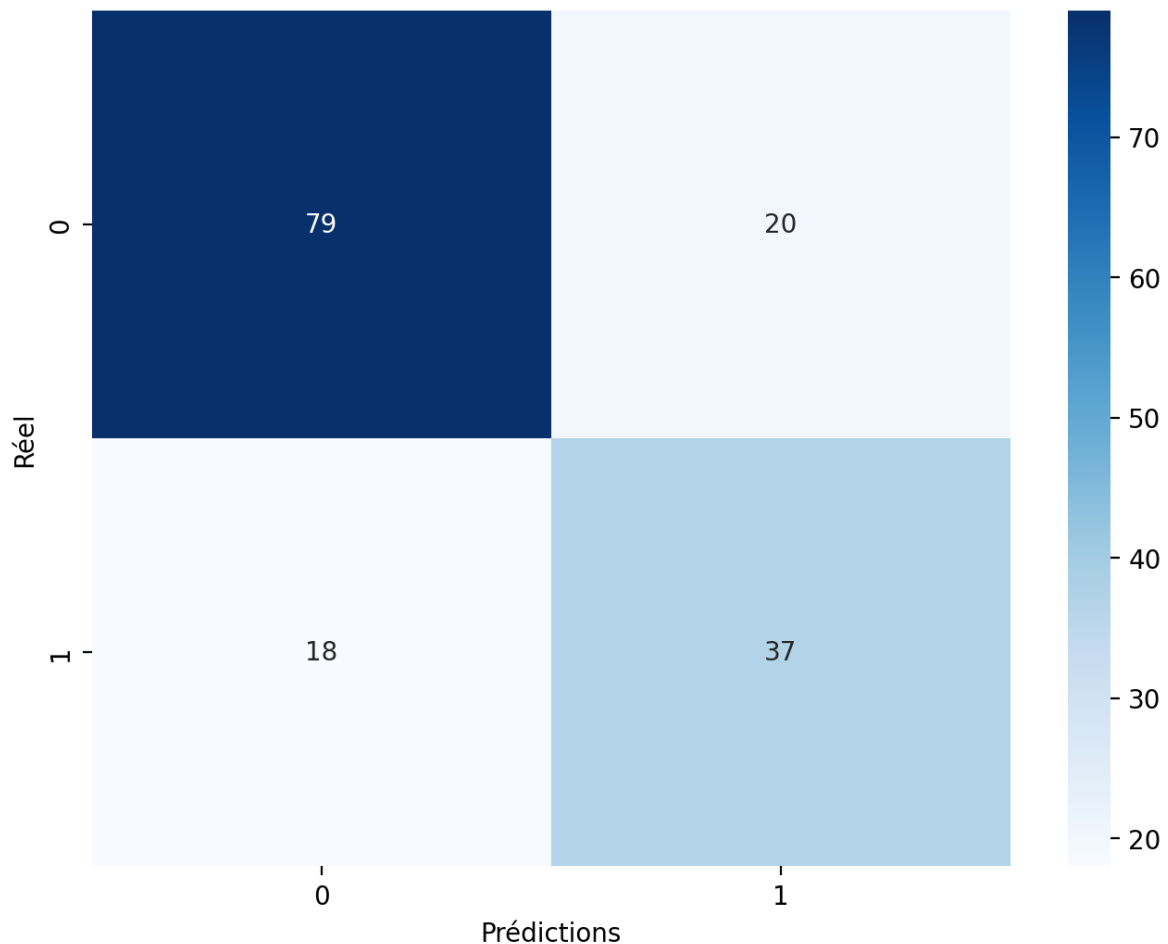


FIGURE 5.14 – Résultats de modèle de classification : Matrice de Confusion

5.13 La page de modélisation (Clustering)

Le dataset *Iris* a été utilisé pour le clustering en considérant uniquement les variables descriptives, sans utiliser la variable cible.

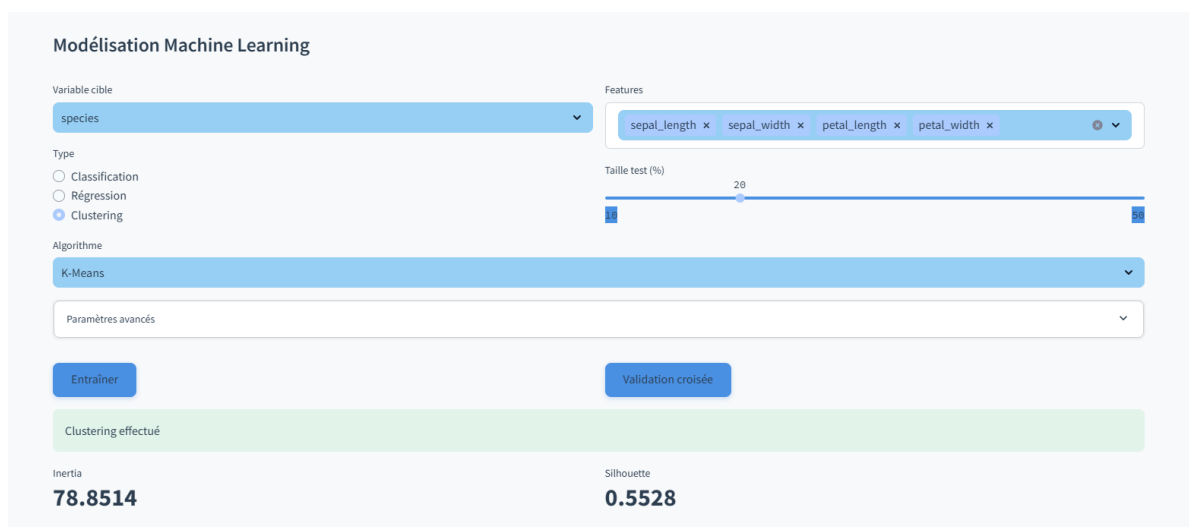


FIGURE 5.15 – La page de modélisation(Clustering)

5.14 Résultats du évaluation (clustering)

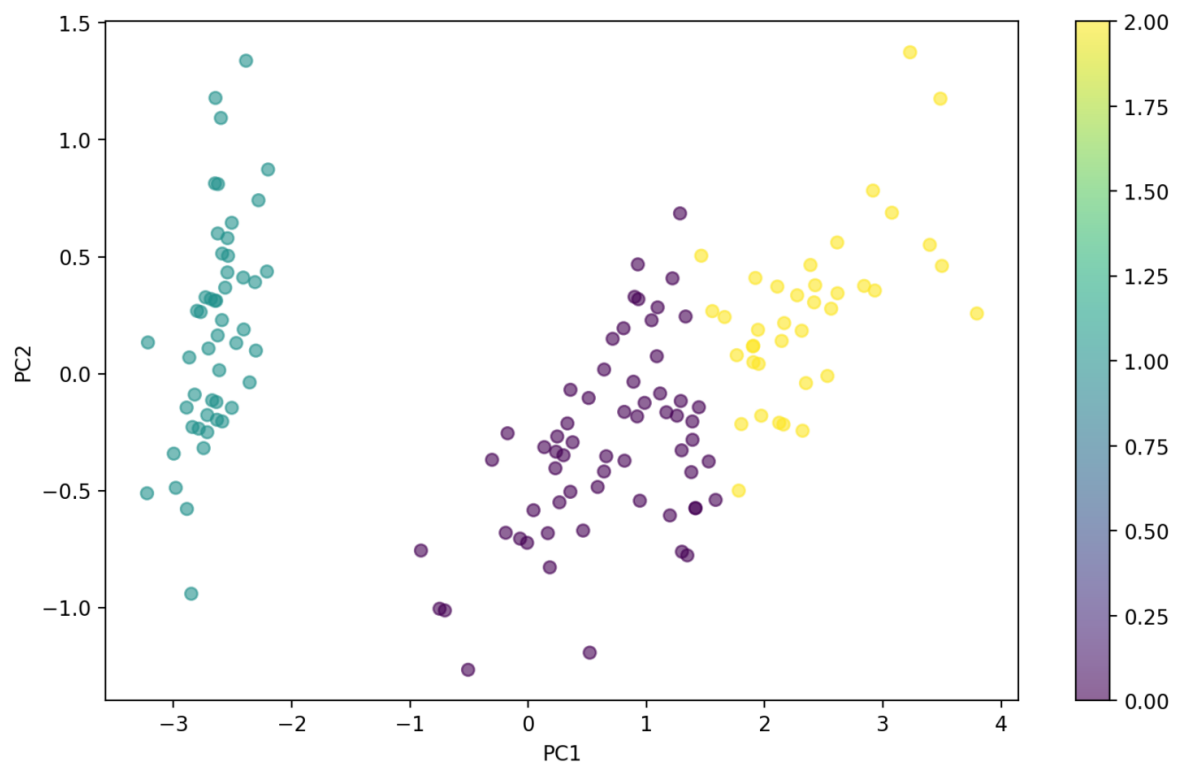


FIGURE 5.16 – Visualisation des clusters obtenus par l’algorithme K-Means

5.15 Exportation des résultats du modèle

Export des résultats du modèle

Cette section permet d'exporter les résultats produits par le modèle entraîné : métriques de performance, prédictions, rapports complets et sauvegarde du modèle.

1. Métriques de performance

Les métriques dépendent du type de problème traité (classification, régression ou clustering).

[Télécharger les métriques \(CSV\)](#) [Télécharger les métriques \(JSON\)](#)

2. Résultats de prédiction

Ce fichier contient les prédictions générées par le modèle. Pour les problèmes de régression, les erreurs sont également fournies.

[Télécharger les prédictions \(CSV\)](#)

3. Sauvegarde du modèle entraîné

Le modèle entraîné peut être sauvegardé afin d'être réutilisé ultérieurement sans relancer l'entraînement.

Nom du fichier du modèle

[Sauvegarder le modèle](#)

FIGURE 5.17 – Interface d’exportation des résultats du modèle

Comme illustré dans la Figure 5.17, l’interface d’exportation permet à l’utilisateur de télécharger les métriques de performance aux formats **CSV** et **JSON**, ainsi que

les résultats de prédiction au format **CSV**. Pour les problèmes de régression, les erreurs associées aux prédictions sont également incluses.

L'interface offre également la possibilité de sauvegarder le modèle entraîné afin de le réutiliser ultérieurement sans relancer la phase d'entraînement.

5.16 Documentation et tutoriels

5.16.1 Tutoriels des algorithmes

La partie *Tutoriels* propose des guides pratiques pour chaque algorithme de Machine Learning disponible dans l'application. Ces tutoriels décrivent le principe général de chaque algorithme et expliquent les étapes nécessaires à son utilisation à travers l'interface.

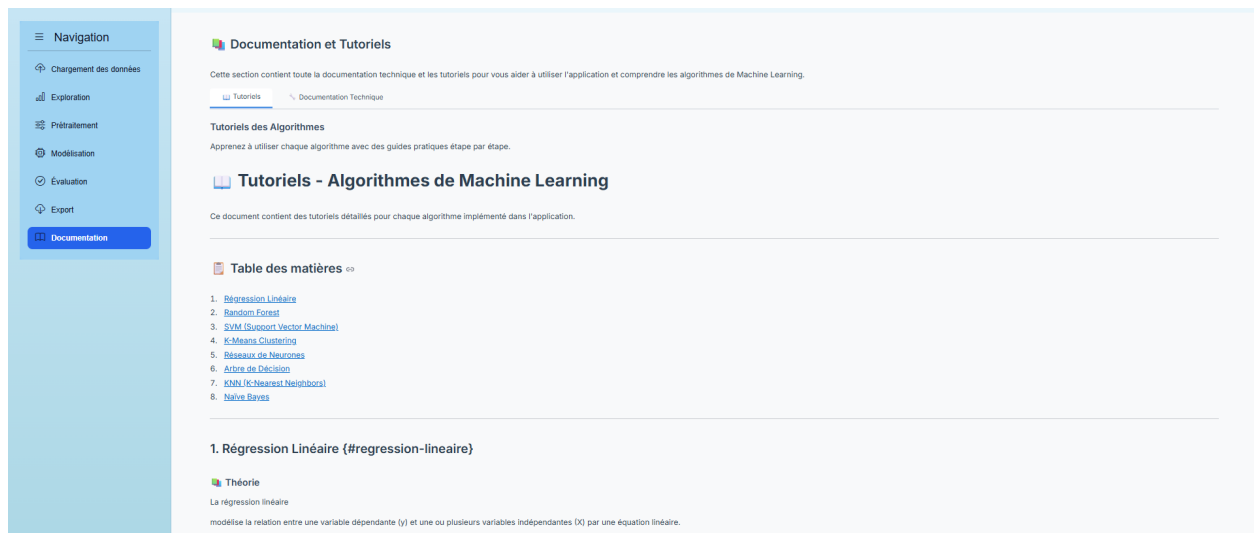


FIGURE 5.18 – Section des tutoriels des algorithmes de Machine Learning

5.16.2 Documentation technique

La documentation technique est destinée aux développeurs et aux contributeurs. Elle présente l'architecture générale de l'application ainsi que la description détaillée des différents modules Python, notamment `data_loader.py`, `preprocessing.py`, `models.py`, `evaluation.py` et `app.py`.

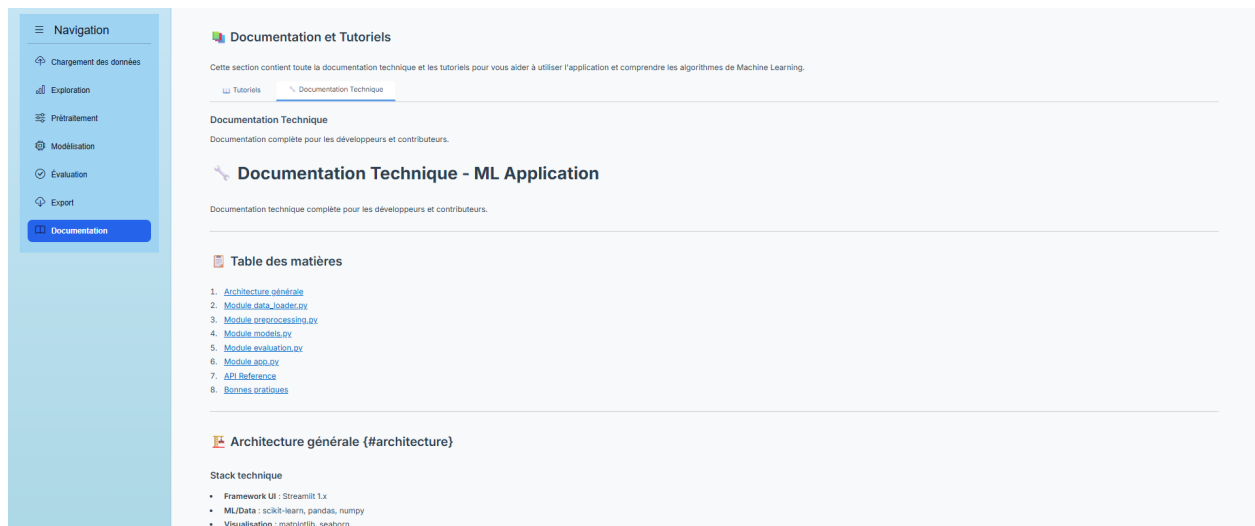


FIGURE 5.19 – Documentation technique de l'application

Chapitre 6 : Conclusion générale et perspectives

6.1 Conclusion générale

Ce projet avait pour objectif principal de **mettre en pratique la programmation avancée en Python** à travers le développement d'une application interactive et modulaire dédiée à l'analyse de données. L'accent a été mis sur la conception logicielle, la structuration du code, la gestion du flux applicatif et l'intégration cohérente de bibliothèques Python.

L'application développée couvre l'ensemble du pipeline de traitement des données, depuis le chargement et le prétraitement jusqu'à la visualisation et l'exportation des résultats. Le **Machine Learning** a été utilisé comme un **support applicatif**, permettant d'illustrer l'utilisation de Python dans un contexte réel, sans viser une optimisation avancée des modèles.

Ce travail constitue ainsi une base solide pour le développement d'applications Python orientées data, maintenables et extensibles, dans un cadre pédagogique.

6.2 Perspectives d'amélioration

Plusieurs perspectives d'amélioration peuvent être envisagées, notamment l'ajout de nouvelles fonctionnalités Python, l'enrichissement de l'interface utilisateur et l'intégration de nouveaux modules afin d'améliorer l'extensibilité de l'application.

D'autres évolutions possibles concernent l'intégration de méthodes automatiques de configuration, l'extension des capacités de visualisation et le déploiement de l'application sur une plateforme cloud, permettant une utilisation à plus grande échelle.